Tema 3.3:

# Generación aumentada por recuperación (RAG) y agentes

Minería de textos, Grado en Inteligencia Artificial (GRIA), Universidad de Vigo
© Prof. David Candal Ventureira - 2026

# Tabla de contenidos


1 Introducción

2 RAG

3 Agentes

# 1

# Introducción

# Introducción: La naturaleza de los LLMs

- Los modelos LLMs generan texto, no información:
  - Su funcionamiento no se basa en técnicas de recuperación de información, que permitan extraer conocimientos factuales de una base de datos.
  - En su lugar, **su generación se basa en patrones lingüísticos** aprendidos durante el entrenamiento:
    - El aprendizaje de patrones de lenguaje a través del entrenamiento sobre textos que incluyen conocimientos, les lleva a generar texto que no sólo es coherente, sino que también responde a las consultas (multidisciplinares) de los usuarios.
    - Los LLMs calculan la afinidad de cada *token* en el vocabulario para continuar el contexto. En la fase de modelado de lenguaje se calcula la distribución de probabilidades y se realiza la selección. Parámetros como la temperatura (“creatividad”) **pueden afectar al grado de cumplimiento con la consulta** del usuario, **e incluso a la veracidad** de la afirmación, independientemente de los patrones de lenguaje aprendidos por el modelo.

# Introducción: La naturaleza de los LLMs

- Los modelos LLMs generan texto, no información:
  - Incluso aunque los patrones de lenguaje aprendidos les lleven a responder texto coherente y que generalmente incluye y desarrolla información factual, **su "conocimiento" está limitado a los textos con los que ha sido entrenado**.
    - El modelo no es capaz (por si mismo) de responder a consultas relacionadas con información posterior a la fecha del documento más reciente con el que ha sido entrenado.
      - No es capaz de "concatenar palabras" que den lugar a una respuesta válida, porque no ha aprendido patrones de ejemplos pertinentes.
      - Reentrenar un modelo adecuadamente es un proceso costoso y largo, **no es viable reentrenar los modelos frecuentemente** (ej: diariamente).
      - Por ello, el modelo no será capaz de responder a informaciones recientes.
  - Adicionalmente, cuando el modelo carece de un dato factual, su naturaleza generativa (determinar el *token* que mejor encaja para continuar el contexto) lo lleva a **"alucinar", en lugar de responder que desconoce la respuesta**.

# Introducción: La naturaleza de los LLMs

```
david@ZephyrusG14:~$ ollama run llama3.1
>>> Háblame de David Candal
David Candal es un futbolista portugués. Nació el 3 de diciembre de 1982 en Lisboa, Portugal. Ha jugado como
delantero y ha pasado por varias equipos, destacando su trayectoria en clubes como el Estoril Praia, el SC Olhão y
el Benfica B, entre otros.

Candal es conocido por sus buenas habilidades para marcar goles y su capacidad de jugar tanto en la delantera
central como por las bandas. Aunque no ha tenido una carrera profesional demasiado destacada a nivel internacional
o en la élite del fútbol portugués, ha sido un valioso jugador en los equipos donde ha jugado.

Si bien no tengo información específica sobre actualizaciones recientes de su carrera o logros posteriores a mi
última actualización, el período desde entonces ha sido utilizado para actualizar y refinar la información
disponible.
```

```
david@ZephyrusG14:~$ ollama run llama3.1
>>> Háblame de David Candal
David Candal es un futbolista español, nacido el 4 de septiembre de 1998. Es conocido por jugar como portero y ha
sido parte del equipo del Real Madrid Castilla (equipo filial del Club de Fútbol Madrid) en la Segunda División de
España.

>>> Háblame de David Candal
Lo siento, pero parece que no tengo información disponible sobre un individuo llamado "David Candal". Es posible
que sea una persona pública menor o que no tenga una presencia notable en los medios de comunicación, lo que hace
que mi conocimiento sobre él sea limitado.

Si me proporcionas más contexto o detalles sobre quién es David Candal y qué hizo notar su nombre, estaré
encantado de intentar investigar más a fondo.

>>> Háblame de David Candal
Después de realizar una nueva búsqueda, encontré que "David Candal" es un entrenador de fútbol español que
actualmente dirige al Real Club Deportivo Femenino La Calahorra en la Segunda División B femenina española.
```

# Introducción: La naturaleza de los LLMs

```
david@ZephyrusG14:~$ ollama run gemma4:26b
>>> ¿Quién es David Candal Ventureira?
Thinking...
*   Subject: David Candal Ventureira.
    *   Language: Spanish.
    *   Goal: Identify who this person is.

    *   Search internal knowledge for "David Candal Ventureira".
    *   *Result:* No specific, widely known public figure, celebrity, politician, or historical figure by this
exact name appears in the training data (up to Jan 2025).
    *   *Note:* Names like this might belong to professionals (lawyers, doctors, academics, engineers) who are not
"famous" in a general knowledge sense, or it could be a private individual.

    *   If the person is not a public figure, I must state that I don't have information about them.
    *   I should avoid guessing or hallucinating.

    *   *Draft 1 (Internal Monologue):* I don't know who that is. He doesn't seem to be a famous person.
    *   *Draft 2 (More professional):* No tengo información específica sobre una persona pública llamada David
Candal Ventureira en mi base de datos.
    *   *Draft 3 (Checking if there's a slight misspelling or context):* Maybe it's a professional? I'll just say
I don't have information.

    *   "No tengo información detallada sobre una figura pública o personaje histórico con el nombre **David
Candal Ventureira** en mis registros. Es posible que se trate de un profesional, un académico o una persona de
ámbito privado que no cuenta con una presencia pública de gran relevancia en los datos con los que fui entrenado."

    *   Does the response comply with the persona (Gemma 4)? Yes.
    *   Is it accurate? Yes (I don't know him).
...done thinking.

No tengo información detallada sobre una figura pública, personaje histórico o celebridad con el nombre **David
Candal Ventureira** en mis registros.

Es posible que se trate de un profesional, un académico o una persona de ámbito privado que no cuenta con una
presencia pública de gran relevancia en los datos con los que he sido entrenado. Si tienes más contexto sobre su
profesión o el ámbito en el que se desempeña, podría intentar ayudarte a buscar información relacionada.
```

# Introducción: La naturaleza de los LLMs

```
david@ZephyrusG14:~$ ollama run gemma4:26b
>>> Háblame de David Candal
Thinking...

*   Subject: David Candal.
*   Language: Spanish.
*   Goal: Provide a comprehensive overview of who he is.

    *   *Who is David Candal?* He is a well-known figure in the Spanish/European automotive and lifestyle content
creation scene, primarily on YouTube and social media.
    *   *Key niche:* Cars (reviews, luxury, modifications, lifestyle).
    *   *Affiliations/Context:* Often associated with high-end car culture, "car influencers," and sometimes
linked to specific automotive media or personal brand content.
    *   *Style:* Enthusiastic, visually polished, focuses on "dream cars," performance, and the lifestyle
surrounding premium vehicles.

    To provide a good answer, I should cover:
    *   Identity (Who he is).
    *   Content/Niche (What he does).
    *   Platform/Reach (Where he is found).
    *   Style/Impact (How he presents himself).

    *   *Introduction:* Define him as a digital creator and automotive influencer. Mention his focus on cars.
    *   *Content Details:*
        *   Reviews of luxury and sports cars.
        *   Focus on "automotive lifestyle."
        *   Sometimes involves technical aspects but mostly the emotional/aesthetic appeal of cars.
    *   *Presence:* YouTube, Instagram, TikTok.
    *   *Key Characteristics:* High production value, passion for motorsports/luxury, connection with the
Spanish-speaking automotive community.
```

# Introducción: La naturaleza de los LLMs

```
    *   *Initial thought:* Is he an engineer? *Correction:* While he knows cars, he is primarily a *content
creator/influencer*, not necessarily an engineer (unless there's a specific private bio I'm missing, but his
public persona is media-focused).
    *   *Note on "Davicandal":* His handle is usually "Davicandal".

    *   *Title:* David Candal: Creador de contenido automotriz.
    *   *Points:*
        1.  **Perfil Principal:** Influencer y YouTuber especializado en el sector automotriz.
        2.  **Tipo de Contenido:** Pruebas de coches (reviews), superdeportivos, estilo de vida, tecnología
automotriz.
        3.  **Estilo Visual:** Cinematográfico, de alta calidad, enfocado en la emoción de conducir.
        4.  **Impacto:** Referente para entusiastas de los coches en España y Latinoamérica.

    (This matches the provided good response).
...done thinking.

**David Candal** es un conocido creador de contenido, influencer y comunicador especializado en el sector
**automotriz**. Se ha ganado una gran relevancia en el ámbito digital (YouTube, Instagram y TikTok) gracias a su
enfoque en los coches de alta gama, superdeportivos y el estilo de vida asociado al automovilismo.

Aquí te detallo los puntos clave para entender quién es y qué hace:

### 1. Especialización en Automoción
Su contenido gira principalmente en torno a la pasión por los motores. No se limita solo a la parte técnica, sino
que se enfoca en la **experiencia de conducción** y la estética de los vehículos. Sus vídeos suelen incluir:
*   **Reviews de superdeportivos:** Pruebas de vehículos de marcas como Ferrari, Lamborghini, Porsche, etc.
*   **Lifestyle (Estilo de vida):** Muestra el lujo, los eventos de automovilismo y la cultura que rodea a los
coches de colección.
*   **Novedades del sector:** Información sobre lanzamientos de nuevas tecnologías y modelos de marcas premium.

### 2. Estilo de Contenido
A diferencia de los canales de mecánica pura o ingeniería, David Candal utiliza un estilo mucho más
**cinematográfico y visual**. Sus producciones suelen tener una alta calidad de imagen y edición, buscando
transmitir la emoción, el sonido del motor y el diseño del coche para cautivar a una audiencia que busca
entretenimiento y aspiración.

### 3. Presencia en Redes Sociales
Es una figura clave en la comunidad de "car influencers" de habla hispana. Su capacidad para conectar con una
audiencia joven y entusiasta le ha permitido colaborar con marcas importantes del sector y participar en eventos
internacionales de automoción.
```

# Introducción: La naturaleza de los LLMs

- Los modelos LLMs generan texto, no información:
  - Por otro lado, aunque los modelos públicos (*open-source* o comerciales) son generalistas (dan respuesta a cuestiones multidisciplinares), **la Industria muchas veces necesita trabajar sobre fuentes de datos privadas y actualizadas**, y **no se puede permitir que el modelo "alucine" y genere texto no veraz**.
  - El razonamiento extendido ("forzar" al modelo a escribir un plan de acción acorde a la consulta de usuario) ha demostrado lograr que los modelos generen respuestas más adecuadas y completas, al sesgar al modelo hacia el dominio y los objetivos que la respuesta debe cubrir.

- La solución pasa por **integrar el texto que incluya la información sobre la cual queremos trabajar en el propio *prompt***, una técnica que se conoce como **generación aumentada por recuperación (RAG)**:
  - Se incluyen en el propio *prompt* los **patrones y datos** que necesitamos.
  - Sesgamos al modelo hacia el **dominio** en el que queremos trabajar.

# 2

# RAG

# RAG: RAG clásico

- RAG **extrae información** relevante de una base de datos externa **basándose en la petición del usuario**, y la **inyecta como contexto adicional dentro de esa misma petición** antes de enviarla al modelo para generar una respuesta.
- El mecanismo de RAG clásico sigue el siguiente procedimiento:
  - El usuario realiza una **consulta**.
  - Antes de utilizar el modelo LLM, **se recuperan los k elementos más relevantes para la consulta** a través de una base de datos vectorial (búsqueda híbrida, RRF).
  - **Se construye el *prompt* real** que se usa contra el modelo LLM, **a través de una plantilla predefinida**:
    - Instrucciones (identidad y objetivo del modelo, en el prompt del sistema).
    - *Chunks* recuperados.
    - Consulta del usuario.
  - Se realiza la **inferencia** con el modelo LLM usando el *prompt* construido.

# RAG: Ejemplo de generación de un *prompt* en RAG

- Ejemplo de construcción del *prompt* mediante RAG:

  <|system|>
  Eres un asistente corporativo. Responde de forma concisa basándote ÚNICAMENTE en la siguiente información recuperada. No uses tu conocimiento previo.
  <|user|>
  Documentos recuperados mediante RAG:
  [Doc 1]: El servidor Alpha se reinicia los martes.
  [Doc 2]: El protocolo de seguridad requiere VPN.
  Consulta: ¿Cuándo hay mantenimiento en Alpha?
  <|model|>
  Los martes.

- Mediante ajuste por instrucciones (entrenamiento), el modelo aprende que debe utilizar los documentos recuperados por RAG e introducidos en el *prompt* para generar la respuesta.

# RAG: El tamaño del *prompt*

- Los modelos actuales tienen ventanas de contexto muy grandes (ej: 128k *tokens* en llama 3.1 7B). Sin embargo:
  - La caché KV crece rápidamente con cada *token* (O(N)), tenemos una matriz K y una matriz V por cada cabeza de atención, que almacena los *embeddings* de cada *token* procesado). Puede llegar a saturar la (V)RAM disponible.
  - La latencia TTFT (*prefill*) y coste por inferencia (*decode*) crecen conforme aumenta el tamaño de la caché KV.
  - Por tanto, **es interesante comprimir la información inyectada mediante RAG** (añadir sólo los textos verdaderamente relevantes).

- **Problema *lost in the middle***: Los modelos Transformer tienden a **ofrecer más peso a la información situada en los extremos del *prompt***, debido a la arquitectura del mecanismo de atención y a la manera en la que son entrenados (artículo):
  - **Efecto primacía**: El texto al inicio recibe mayor atención (generalmente instrucciones).
  - **Efecto recencia**: El texto al final recibe mayor atención (generalmente, última consulta).

# RAG: Recuperación de información

- La recuperación de información en RAG requiere:
  - **Segmentación** de documentos en *chunks*.
  - Uso de **bases de datos vectoriales para la recuperación** de *chunks* relevantes.
  - Uso de estrategias de **búsqueda híbrida** (semántica y léxica) y mecanismos de fusión como RRF.
  - Uso de estrategias de **búsqueda aproximada** (ANN) como HNSW, para permitir recuperar, con una latencia muy baja, un conjunto de textos relevantes para la consulta dentro de un *dataset* de información masivo.

- Fases de recuperación:
  - **Fase 1: Recuperación**: Usamos búsqueda híbrida para la recuperación rápida e imprecisa de documentos potencialmente relevantes (ej: 10M -> 100).
  - **Fase 2: *Reranking***: Usamos un modelo cross-encoder para evaluar la relevancia de cada documento con la consulta del usuario (preciso pero lento). Filtramos los documentos de la fase 1 para quedarnos con los k más relevantes (ej: 100 -> 5).

# RAG: Recuperación de información

- El proceso de *chunking* de los documentos locales que se usarán para RAG es relevante:
  - ***Chunks* demasiado grandes** generan *embeddings* ruidosos (combinan demasiada información semántica).
  - ***Chunks* demasiado pequeños** generan *embeddings* más precisos, pero pueden eliminar información relevante para la consulta (dividir una frase o párrafo en 2 *chunks* independientes, que en conjunto tienen mucha relevancia semántica con la consulta del usuario), o eliminar contexto sobre la información que es relevante para que el LLM genere una respuesta adecuada.
    - Nota: El problema principal es no poder recuperar documentos adecuados en base a la consulta, dado que podemos inyectar a posteriori en el prompt los *chunks* adyacentes al recuperado.

- Estrategias de *chunking*:
  - **Segmentación por tamaño fijo con superposición** entre *chunks*.
  - **Basado en estructura** (ej: separadores como saltos de línea o etiquetas de estructura en HTML, Markdown, etc.).
  - **Basado en modelos de PLN** (segmentar en base a los cambios de temática de los textos).

# RAG: Límites de los sistemas RAG y complementos

- El enfoque RAG clásico permite construir respuestas en base a información que el modelo LLM no ha visto durante su entrenamiento. Sin embargo:
  - **Es un modelo rígido**: Se realiza una consulta en la DB vectorial para cualquier *prompt* del usuario, incluso para aquellos que no requieren búsqueda de información.
  - **Es un modelo simple**: No es capaz de evaluar (más allá de la coincidencia léxica y semántica) si los textos recuperados responden a la consulta del usuario, ni detectar si es necesario emplear una estrategia de búsqueda alternativa o reformular la pregunta del usuario.
- Estos problemas serán corregidos mediante la introducción de arquitecturas agénticas…

# RAG: Límites de los sistemas RAG y complementos

- Estos problemas serán corregidos mediante la introducción de arquitecturas agénticas…
- Pero podemos mejorar el rendimiento del RAG clásico a través de mecanismos como:
  - **Reformulación de la consulta (*query rewriting / query expansion*)**: Usamos un LLM sencillo para generar una consulta de mayor calidad.
  - ***Multi-query retrieval***: De manera similar, se generan 3-5 versiones de la consulta, bajo diferentes enfoques. Sus resultados se combinan y se hace *reranking* con un modelo cross-encoder.
  - **HyDE (*hypothetical document embeddings*)**: En lugar de usar la consulta del usuario para la recuperación de información, se utiliza un modelo LLM sencillo para generar una respuesta hipotética. La respuesta puede no ser correcta, pero sigue el formato e incluye las palabras claras del texto que se quiere extraer.

# RAG: Límites de los sistemas RAG y complementos

- Estos problemas serán corregidos mediante la introducción de arquitecturas agénticas…
- Pero podemos mejorar el rendimiento del RAG clásico a través de mecanismos como:
  - **Reranking del contexto**: La búsqueda de información nos devuelve los *chunks* ordenados por relevancia, pero sabemos que el modelo hace que los términos presten mayor atención a los textos en los extremos (LitM) -> Reordenamos los *chunks* (ej. pasamos de [1,2,3,4,5] a [1,3,5,4,2]).
  - **Caché semántica**: En Industria, una cantidad significativa de consultas tienen siempre la misma respuesta -> Guardamos en la DB vectorial la respuesta y, si la coincidencia semántica es muy alta, respondemos con ese contenido (no se usa el LLM).

# RAG: Límites de los sistemas RAG y complementos

- Ejemplo de *multi-query retrieval*:
  - [Usuario]: “no va la pantalla”
  - [Modelo simple]: [“pantalla encendida negro”, “fallo pantalla”, “no hay señal”, “no video output”]
  - Se recuperan 100 *chunks* para cada consulta generada, y se combinan.
  - Se hace *reranking* de los *chunks* para determinar los 5 más relevantes a la consulta.
  - Se genera el *prompt* del usuario, incluyendo estos documentos.

- Ejemplo de HyDE:
  - [Usuario]: “¿Qué es el mecanismo HyDE es los modelos LLM?”
  - [Modelo simple]: “**HyDE es un mecanismo** de auto-censura semántica que detecta si el **modelo LLM** está ‘empezando a aburrirse’ y le inyecta pequeñas dosis de sarcasmo para mantenerlo alerta.”
  - Se recuperan documentos en base a la respuesta generada por el modelo simple.

# RAG: Evaluación de sistemas RAG

- Para evaluar un sistema RAG completo, debemos prestar atención a:
  - **Documentos recuperados** (presencia de documentos relevantes y *ranking*).
  - Validar que el modelo se ha basado **exclusivamente en la información incluida** en los documentos proporcionados.
  - Validar si el **modelo resuelve la consulta** del usuario.

- Mecanismos de evaluación:
  - ***LLM-as-a-judge***: Enviamos la consulta, los documentos recuperados y la respuesta a un LLM para que evalúe el resultado.
  - ***RAGAS (RAG assessment)***: *Framework* basado en *LLM-as-a-judge*, evalúa el sistema midiendo:
    - **Relevancia**: Mide si la respuesta contesta a la consulta.
    - **Precisión y *recall***: Mide el ratio de documentos recuperados que son útiles para la consulta, y el grado en el que la información que requiere la consulta está presente en ellos.
    - **Fidelidad**: Mide si la respuesta está respaldada por los documentos recuperados.

# RAG: GraphRAG

- La recuperación mediante búsqueda híbrida (semántica + léxica) permite encontrar documentos que contienen hechos aislados relevantes a la consulta.
- Sin embargo, **no es una solución óptima cuando** necesitamos sintetizar información en base a **conocimientos que se conectan a través de relaciones**.

- **GraphRAG** construye un grafo en el que los **nodos (entidades)** están conectados a través de **relaciones (verbos)**. Esto nos permite construir un árbol de relaciones que nos aporte contexto para la generación de texto.
- El procedimiento de búsqueda es como sigue:
  - Se realiza una **búsqueda semántica del nodo de entrada** en el grafo (similitud coseno con la consulta).
  - **Extraemos caminos de n saltos**, a través de las relaciones entre los nodos.
  - **Introducimos los caminos descubiertos en el contexto**.

# RAG: GraphRAG

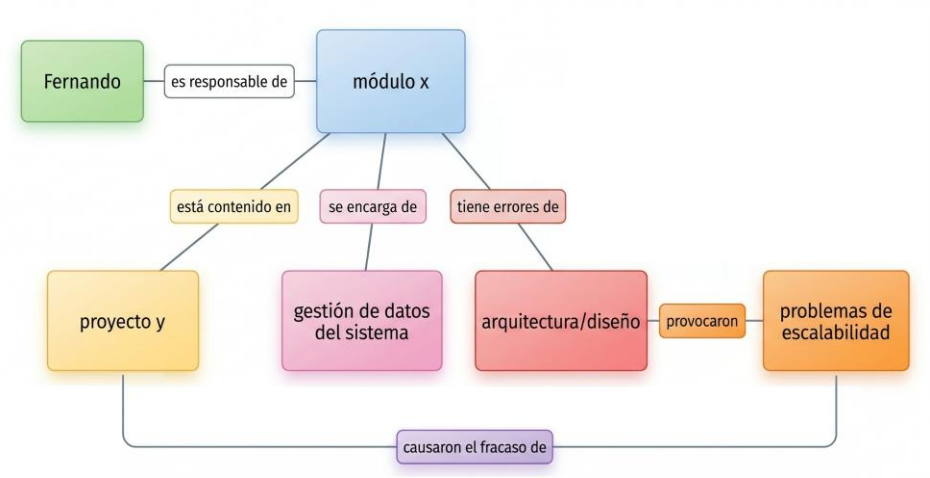
- Ejemplo con RAG:
  - Doc 1: “Fernando es el responsable del módulo X dentro del proyecto Y, encargado de la gestión de datos del sistema.”
  - Doc 2: “El proyecto Y ha fracasado debido a problemas graves de escalabilidad.”
  - Doc 3: “El módulo X presentó errores de diseño en su arquitectura, lo que provocó problemas de escalabilidad en el sistema.”

  - Consulta: ¿Quién es el responsable del fracaso del proyecto Y?
  - Documentos recuperados: Doc 2.
  - Respuesta: Problemas graves de escalabilidad.

- Podemos obtener la respuesta que buscamos encadenando consultas.

# RAG: GraphRAG

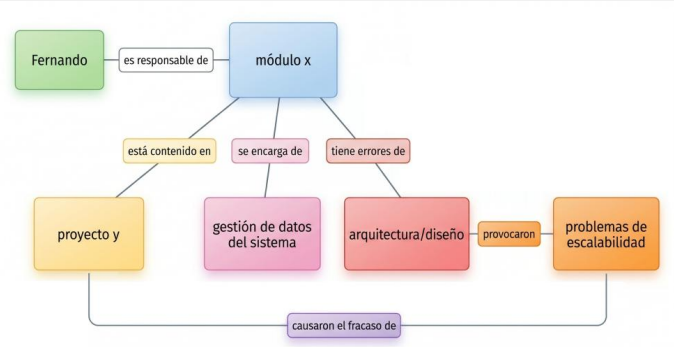
- Ejemplo con GraphRAG:




- Consulta: ¿Quién es el responsable del fracaso del proyecto Y?
- GraphRAG: Proyecto Y -> fracasa por -> Problemas de escalabilidad -> provocados por -> errores de arquitectura/diseño -> que tiene el -> módulo X -> que es responsabilidad de -> Fernando.

# RAG: GraphRAG

- Ejemplo con GraphRAG:
  - Consulta: ¿Quién es el responsable del fracaso del proyecto Y?
  - GraphRAG: Proyecto Y -> fracasa por -> Problemas de escalabilidad -> provocados por -> errores de arquitectura/diseño -> que tiene el -> módulo X -> que es responsabilidad de -> Fernando.
  - Respuesta: “Según los registros del grafo, el responsable es **Fernando**. Esto se debe a que **Fernando** es el encargado del **módulo X**, el cual presentó **errores de arquitectura** que derivaron en **problemas de escalabilidad**, siendo estos últimos la causa directa del fracaso del **Proyecto Y**.”



# 3

# Agentes

# Agentes: Introducción

- Los sistemas basados en LLMs tradicionales son demasiado simples, siguen un flujo lineal consulta -> respuesta:
  - Incapaces de condicionar la generación de texto.
  - Incapaces de realizar acciones externas (ejecutar código para observar el resultado, utilizar APIs de red para consultas/acciones, etc.).
  - Incapaces de analizar su propia generación, detectar errores y repetir la inferencia.

- En los **sistemas agénticos basados en LLMs**, el modelo deja de ser una simple herramienta de generación de texto para formar parte de un sistema que:
  - **Planifica** las acciones que se deben realizar.
  - **Realiza** las acciones (herramientas externas).
  - **Observa** los resultados.
  - **Itera** sobre el problema.
  - **Genera una respuesta**.

# Agentes: Arquitecturas agénticas

- Tipos de **arquitecturas agénticas**:
  - **ReAct (*Reason + Act*)**:
    - El orquestador **delega totalmente la gestión de la generación a un modelo LLM**.
    - A través del razonamiento extendido, el modelo identifica el problema existente en base al contexto, determina una acción (ejecutada por el orquestador), observa su resultado y genera una respuesta.
    - El bucle se repite hasta que el modelo determina que ha satisfecho la consulta del usuario.
    - El plan se ajusta sobre la marcha en base a los resultados.
      - Ventaja: El **sistema se autoajusta** a las observaciones.
      - Problema: Puede entrar en un **bucle infinito** y ser incapaz de alcanzar una respuesta, o determinar que ha alcanzado la condición de parada en un instante no adecuado.

# Agentes: Arquitecturas agénticas

- Tipos de **arquitecturas agénticas**:
  - **ReAct (*Reason + Act*)**:

```
# PROMPT DE SISTEMA
Responde a la siguiente pregunta usando estas herramientas: [WebSearch, Calculator].

Usa ESTRICTAMENTE el siguiente formato:
Question: la pregunta de entrada a responder
Thought: siempre debes pensar qué hacer
Action: la acción a tomar, debe ser una de [WebSearch, Calculator]
Action Input: el input de la acción
Observation: el resultado de la acción
... (Thought/Action/Action Input/Observation puede repetirse N veces)
Thought: ahora sé la respuesta final
Final Answer: la respuesta final a la pregunta
```

# Agentes: Arquitecturas agénticas

- Tipos de **arquitecturas agénticas**:
  - **Grafo dirigido acíclico (DAG)**: El sistema se organiza a través de un **grafo de decisión fijo** y sin ciclos (actualmente se implementan ciclos controlados):
    - Cada **nodo** representa una **responsabilidad única y aislada** (LLM, herramienta, validación, etc.).
    - El **flujo de ejecución está predefinido y estructurado**. El sistema determina los caminos a utilizar en función de la consulta.
    - Se pueden **ejecutar múltiples nodos en paralelo** (ej: inferencia sobre dos textos diferentes), si la salida de un nodo son múltiples caminos.
    - Permite **incorporar evaluación y validación explícita** dentro del flujo tras un nodo.
    - Podemos disponer de **diferentes nodos LLM, con contextos independientes o compartidos**.

      - Ventaja: **Mayor control, trazabilidad**, paralelismo y facilidad de depuración.
      - Problema: **Menor flexibilidad**, dificultad para adaptarse a situaciones no previstas en el grafo.

# Agentes: Arquitecturas agénticas

- Tipos de **arquitecturas agénticas**:
  - **Grafo dirigido acíclico (DAG)**: El sistema se organiza a través de un grafo de decisión fijo y sin ciclos:
    - **Tipos de nodos en un DAG**:
      - **Agente**: Nodo vinculado a un modelo LLM, que tiene como objetivo razonar en base al contexto disponible y seleccionar una acción (ej: uno o varios de los caminos disponibles).
      - **Cadena**: Un nodo vinculado a un modelo LLM que tiene como objetivo generar un texto de salida en base a un texto de entrada.
      - **Herramienta**: Función específica. En base a parámetros de entrada, ejecuta una acción (API de red, cálculo…) y devuelve el resultado.

# Agentes: Decodificación restringida

- Las herramientas que ejecuta el sistema agéntico requieren una serie de argumentos de entrada (en general, un JSON o YAML) -> El modelo LLM debe **garantizar que el formato de salida de este texto** es el adecuado.

- Opciones:
  - **Bucle de validación**: Comprobamos el formato del texto y repetimos la generación en caso de que no sea el adecuado. Puede generar ciclos infinitos, especialmente si el objeto es grande.
  - **Decodificación restringida**: Alteramos la distribución de probabilidades de los *tokens* a seleccionar durante la inferencia, de manera que la salida se restringe a aquellos permitidos (se establece a -∞ los *tokens* no permitidos, antes de aplicar softmax).
    - Ej: Si el contexto es "… 'edad': ", los *tokens* permitidos son aquellos que incluyan uno o más enteros ([0-9]+). Los logits de cualquier otro *token* deben establecerse a -∞.

# Agentes: Sistemas RAG agénticos

- Un **sistema agéntico basado en DAG para RAG** tiene una arquitectura equivalente a la siguiente:
  1. **Agente / *router***: Se encarga de la toma de decisiones en las posibles bifurcaciones del grafo (ej: selección de las herramientas a utilizar).
  2. (Opcional) **Herramienta *multi-query retrieval***: Genera consultas diversas en base a la consulta del usuario.
  3. **Herramienta de recuperación**: Obtiene los documentos relevantes para una consulta (si existen múltiples consultas, se pueden ejecutar en múltiples instancias en paralelo).
  4. **Herramienta de agregación y *reranking***: Recoge los documentos relevantes y los ordena por afinidad a la consulta (cross-encoder o LLM).
  5. **Cadena de síntesis**: Formula la respuesta en base a la información recuperada.

  El agente coordina y valida las interacciones entre nodos. Determina si los **documentos recuperados son suficientes** para formular la respuesta, si es necesario **recurrir a una herramienta adicional** (ej: búsqueda WEB) o **repetir la recuperación con otros argumentos**. Puede recurrir a bucles controlados.

# Agentes: Sistemas multi-agente

- En los sistemas agénticos con un único agente, el agente es generalista, está configurado con instrucciones generales (*prompt* de sistema) que le permiten ajustar sus decisiones al estado actual de la ejecución (aunque puede variar su contexto durante la ejecución).
- En los **sistemas multi-agente**, el grafo se divide en **áreas disjuntas**, en base a su **objetivo y a las herramientas** que se utilizan en su interior.
  - **Cada agente está especializado** en un conjunto de acciones y herramientas disponibles acotado (*prompt* de sistema más simple).
  - Cuando el flujo pasa de un área a otra, **el agente del área saliente genera un informe adaptado a las necesidades de la nueva área**. El **nuevo agente se basa exclusivamente en esta información** en lugar de utilizar el contexto completo desde el inicio (reduce saturación del contexto).
- Ejemplos de agentes:
  - **Investigador**: Optimizado para RAG y búsqueda WEB.
  - **Programador**: Estricto en el formato de salida y la lógica del código.
  - **Revisor**: Detecta alucinaciones en la salida.

# Agentes: Arquitecturas multi-agente

- Las **principales arquitecturas multi-agente** son:
  - **Secuencial**: Los agentes se concatenan secuencialmente, de manera que la salida de uno es la entrada del siguiente.
  - **Jerárquico**: Un agente líder se encarga de descomponer la tarea y asigna cada subtarea a un agente especializado. El agente líder recoge las respuestas del resto de agentes, y las procesa y combina para utilizarlas en nuevas subtareas.
  - **Colaborativo**: Varios agentes se comunican entre sí en un entorno compartido, con el objetivo de alcanzar una solución conjunta (generan respuestas que los demás agentes procesan para determinar una solución). En una arquitectura completamente colaborativa, los propios agentes determinan la siguiente subtarea y el agente o agentes que deben coordinarla.

# Agentes: Seguridad y gobernanza

- Los sistemas agénticos pueden llevar a cabo **acciones con repercusiones críticas**:
  - Eliminar ficheros en un disco duro.
  - Publicar información sensible en Internet.
- **Las reglas de funcionamiento del modelo se introducen como contexto** (*prompt* del sistema), sólo **refuerzan patrones de lenguaje** que fomentan que no se incumplan las reglas del sistema.
- A través del **ajuste por instrucciones**, el modelo ha sido sesgado para no cumplir las instrucciones del usuario en caso de que supongan una amenaza (se separan los contextos de sistema y usuario con diferentes etiquetas y se refuerza la prioridad de las instrucciones en el *prompt* de sistema).

- **Inyección de *prompt* (*prompt injection*)**: Insertar en un *prompt* de usuario, o en documentos y WEBs accesibles por un modelo LLM, una instrucción oculta, con el objetivo de tergiversar la respuesta (y/o acciones) generadas por el modelo.

# Agentes: Seguridad y gobernanza

- Para protegerse contra acciones malintencionadas, los sistemas agénticos recurren a **protecciones a nivel de orquestador**:
  - El orquestador **revisa los textos de entrada y salida de un agente**, utilizando expresiones regulares y otro tipo de reglas deterministas, con el objetivo de intervenir la generación de texto en el caso de que se detecten acciones potencialmente peligrosas.
  - **Barreras de entrada (*input guardrails*)**: Detectan intentos de *jailbreak* o PII que pueda verse comprometida.
  - **Barreras de salida (*output guardrails*)**: Verifican que la respuesta no supone un peligro y que no revela datos privados del modelo (ej: el *prompt* de sistema).
  - Adicionalmente, el orquestador guarda un **registro de las acciones que desencadena la consulta** del usuario, con el objetivo de detectar comportamientos adversos derivados de ataques sutiles.

# ¿Preguntas?